
ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 3
[C++ UNIX]: C++ CLI / FUNCTION / LOOP / RECURSION

Группа: Z33433

Студент: Железнова Александра Сергеевна

Преподаватель: Маслов Иван Дмитриевич

1 Цель работы

Познакомить студента с принципами компиляции исходного кода. Составить программу с использованием циклов, условий и функций. Сравнить быстродействие между C++ и Python. Ознакомление с типами данных.

2 [C++ CLI CALC] Создать программу CALC с интерфейсом CLI

```
#include <iostream>
#include <string>
#include <cstdlib>

using namespace std;

// Функция сложения
int add(int a, int b) {
    return a + b;
}

// Функция вычитания
int subtract(int a, int b) {
    return a - b;
}

// Функция возведения в степень (только целые степени)
int power(int base, int exponent) {
    int result = 1;
    bool negative = false;
```

```

if (exponent < 0) {
    // Для простоты ограничимся: степень не может быть отрицательной
    cout << "Ошибка: отрицательная степень не поддерживается." << endl;
    return 0;
}

for (int i = 0; i < exponent; ++i) {
    result = result * base;
}

return result;
}

int main(int argc, char* argv[]) {
    if (argc != 4) {
        cout << "Использование: CALC <чиселко_раз> <оператор> <чиселко_два>" << endl;
        return 1;
    }

    // Преобразуем аргументы
    int a = atoi(argv[1]);
    string op = argv[2];
    int b = atoi(argv[3]);

    // Выбор операции
    if (op == "+") {
        cout << "Результат: " << add(a, b) << endl;
    } else if (op == "-") {
        cout << "Результат: " << subtract(a, b) << endl;
    } else if (op == "^") {
        cout << "Результат: " << power(a, b) << endl;
    } else {
        cout << "Ошибка! неподдерживаемый оператор '" << op << "'." << endl;
        return 1;
    }

    return 0;
}

```

3 [C++ RECURSION] Решить задачу ханойской башни с использованием рекурсии

```

#include <stdio.h>
#include <stdlib.h>

// Структура для элемента двусвязного списка
typedef struct Node {
    int disk;
    char from;

```

```

    char to;
    struct Node* prev;
    struct Node* next;
} Node;

// Глобальные переменные для головы и хвоста списка
Node* head = NULL;
Node* tail = NULL;

// Функция для добавления хода в список
void add_move(int disk, char from, char to) {
    Node* new_node = (Node*)malloc(sizeof(Node));
    new_node->disk = disk;
    new_node->from = from;
    new_node->to = to;
    new_node->prev = tail;
    new_node->next = NULL;

    if (tail == NULL) {
        head = new_node;
        tail = new_node;
    } else {
        tail->next = new_node;
        tail = new_node;
    }
}

// Рекурсивная функция решения Ханойской башни
void hanoi(int n, char from_rod, char to_rod, char aux_rod) {
    if (n == 1) {
        add_move(n, from_rod, to_rod);
        return;
    }
    hanoi(n - 1, from_rod, aux_rod, to_rod);
    add_move(n, from_rod, to_rod);
    hanoi(n - 1, aux_rod, to_rod, from_rod);
}

// Функция для печати списка ходов
void print_moves() {
    Node* current = head;
    int move_num = 1;
    while (current != NULL) {
        printf("Ход %d: Переместить диск %d с %c на %c\n",
            move_num++, current->disk, current->from, current->to);
        current = current->next;
    }
}

// Функция для освобождения памяти списка

```

```

void free_list() {
    Node* current = head;
    while (current != NULL) {
        Node* temp = current;
        current = current->next;
        free(temp);
    }
    head = NULL;
    tail = NULL;
}

int main() {
    int num_disks = 8;

    printf("Решение головоломки Ханойская башня с дисками:\n", num_disks);
    hanoi(num_disks, 'A', 'C', 'B');
    print_moves();

    free_list();
    return 0;
}

```

4 GIT

Опять-таки все как обычно в двух лабах до этого, заливаем все, запускаем скрипты, пушим и живем дальше.